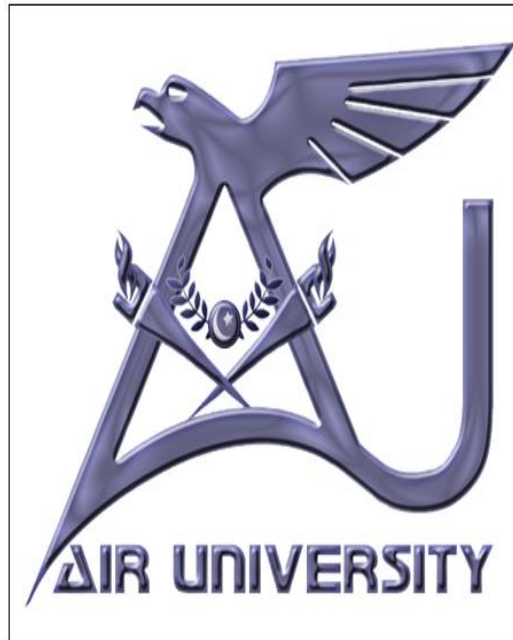




TRAFFIC VIOLATION CHALLAN MANAGEMENT SYSTEM

Group Members:

SHOAIB ZIA (2501559)
HASHIR RAUF (2501586)
ABDULLAH RAMZAN (2501574)

TEAM: HOMO COMPILICUS
BS Mechatronics Engineering

DEPARTMENT OF Mechatronics
FACULTY OF ENGINEERING
AIR UNIVERSITY, ISLAMABAD

2026

Policy on Professional Ethics & Plagiarism

You are free to consult any book and online resources during the design and analysis phase, but you could not copy from them. Your design and implementation must be your original effort and the same applies to the source code. Remember that if anyone is found to copy from the Internet or other group members, the group shall face severe penalty. You are not allowed to copy any material or code directly from the web or elsewhere. Note that if you are found to violate this policy or it becomes obvious that the work you have submitted is not your own or has been taken from some other source you shall be facing severe consequences.

Plagiarism will be tested through software and it must be less than 20%.

Team Contribution Table

Team Member Name	Role	Word Count (Assigned Color)	Signature
Shoaib Zia	ESP32 Developer		
Hashir Rauf	PC App Developer		
Abdullah Ramzan	Docs & Testing		

Chapter 1 — Preliminaries

1.1 Project Proposal

The project involves creating a Traffic Violation Challan Management System using ESP32 microcontroller for Pakistani traffic police, aimed at providing a portable hardware solution for capturing, searching, updating, and deleting traffic violation challans on the field. The ESP32 acts as a hardware authentication dongle and Plate ID input device. A C++ application on the PC end manages a CSV-based database of challan records.

1.2 Initial Feasibility

The project was technically and economically feasible. The ESP32 microcontroller module was selected for its inexpensive cost, its wide use in embedded systems and its capability to communicate through serial at 115200 baud. The program is developed using C++ file input-output via standard C++ library and is capable of storing a few thousand records. The CSV file format allows the database to run on spreadsheet software. The total cost of the hardware is estimated at not more than PKR 2000 making it affordable for any traffic department.

1.3 Team Roles & Details

Member	Roll No.	Responsibilities
Shoaib Zia	2501559	ESP32 firmware
Hashir Rauf	2501586	C++ PC application,
Abdullah Ramzan	2501574	flowcharts, block diagrams, 100 CSV records generation, system testing

1.4 Work Breakdown Structure (WBS)

WBS ID	Task	Assigned To	Status
1.0	Project Planning & Proposal	All Members	Complete
2.0	ESP32 Firmware Development	Shoaib Zia	Complete
2.1	Password Authentication (ESP32)	Shoaib Zia	Complete
2.2	Plate ID Serial Transmission	Shoaib Zia	Complete
3.0	C++ PC Application Development	Hashir Rauf	Complete
3.1	initializeDatabase() Function	Hashir Rauf	Complete
3.2	isUnique() Function	Hashir Rauf	Complete
3.3	appendRecord() Function	Hashir Rauf	Complete

3.4	searchByID() Function	Hashir Rauf	Complete
3.5	updateRecord() Function	Hashir Rauf	Complete
4.0	CSV Database (100+ Records)	Abdullah Ramzan	Complete
5.0	Integration & Testing	All Members	Complete
6.0	Report & Documentation	Abdullah Ramzan	Complete

1.5 Gantt Chart

Task	Wk1	Wk2	Wk3	Wk4	Wk5	Wk6	Wk7
Project Planning	■	■					
ESP32 Development		■	■	■			
C++ App Development			■	■	■		
CSV Database Creation				■	■		
Integration & Testing					■	■	
Report Writing						■	■

1.6 Estimated Budget

Component	Qty	Est. Cost (PKR)
ESP32 Development Board	1	1,200
USB-A to Micro-USB Cable	1	150
Breadboard & Jumper Wires	1 set	200
Dev C++ (Free Software)	—	0
Arduino IDE (Free Software)	—	0
TOTAL		~1,550

Chapter 2 — Project Conception

2.1 Introduction

Currently, Pakistan has a manual traffic challan management system, which is prone to human errors, document loss, and illegal access. This project proposes the development of an ESP32-PC Hybrid Traffic Violation Challan Management System for the generation and management of traffic violation challans on a computer, tablet, or mobile phone.

The system consists of embedded hardware and a desktop software application. The ESP32 microcontroller reads the field level authentication and vehicle plate ID and passes them to the PC application via a serial connection. The PC application is a menu-driven C++ console application. It manages the complete database and saves all entries in a CSV file.

2.2 Project Development Process

The overall procedure used to design this system was done in an iterative development cycle: This has included:

1. Gathering requirements - finding current law enforcement personnel work flows, work conditions on the street, and the data fields needed to be collected.
2. Development of the ESP32 firmware - adding password authentication functions, developing license plate number reading capabilities, and implementing serial transmission of data.
3. Development of the PC application- added all five CRUD operations with a two-levels of authentication.
4. Creation of the CSV database - populated with a dataset containing over 100 real-world records of vehicles registered in Pakistan.
5. Integration testing of all components - connecting both the ESP32 and PC components from start to finish.
6. Preparing documentation and reports.

2.3 System Block Diagram

The system architecture is divided into three layers: the ESP32 Hardware Layer, the PC Software Layer, and the Data Layer. The diagram below illustrates the data flow and component interactions:

[SYSTEM BLOCK DIAGRAM]

ESP32 HARDWARE LAYER	PC SOFTWARE LAYER	DATA LAYER
• Password Input (Keypad/Serial) • Auth Module (Verify Password) • Plate ID Sender (TX via Serial)	• PC Auth Module (Password Verify)	• CSV Database (challan records) • Temp File (used during updates)

	• DB Initializer (initializeDatabase) • Menu Controller (Route User Choice)	• CRUD: Search / Add / Update / Delete
--	--	--

Data Flow: User → ESP32 Auth → Plate ID (Serial) → PC Auth → Menu → CRUD → CSV

2.4 Deliverable Specification Sheet

Parameter	Specification
Microcontroller	ESP32 Dev Module (Xtensa LX6, 240 MHz, 4MB Flash)
Communication	UART Serial @ 115200 Baud over USB-Serial (CP210x)
PC Language	C++ (ISO C++11) compiled with Dev-C++ (MinGW GCC)
Database Format	CSV (Comma-Separated Values), plain text
Database Fields	ChallanID, PlateID, OwnerName, Violation, FineAmt, Date, Status
Records (Pre-loaded)	100+ realistic Pakistani vehicle challan records
Authentication	Double-layer: ESP32 side + PC side, default password: challan123
CRUD Functions	5 mandatory functions fully implemented
Operating System	Windows 10/11 (Windows.h API for COM port access)
Serial Port	COM3 (configurable in verifyHardwareKey() function)

Chapter 3 — Software / Firmware Design

3.1 Controller Selection — ESP32

The ESP32 was chosen as the hardware controller for many reasons:

- Includes a dual core Xtensa LX6 CPU capable of up to 240mhz, which is fast enough for real time serial communications.
- Has built-in hardware UART to support baud rates of up to 5mbps, but we have seen reliable operation at 115200 baud.
- USB to Serial bridge typically CP2102/CH340, which allows direct connection to a PC without the need for additional hardware.
- Very reasonable cost (~PKR 1200) and commonly available in electronics shops across Pakistan.
- Supports Arduino IDE with a wealth of libraries available for fast and easy development.

3.2 Input/Output Pin Configuration

Pin/Interface	Direction	Function	Notes
USB (Serial0)	Output (TX)	Plate ID & key string transmission	115200 Baud
USB (Serial0)	Input (RX)	Challenge byte 'R' reception from PC	Polled
GPIO 0 (Boot)	Input	Flash mode (used during programming)	Pull-up
EN Pin	Input	Module reset	Active High
3.3V / GND	Power	Module power supply	300mA max

3.3 Software Design Details & User Requirements

This program provides five main database management capabilities via a menu driven console application.

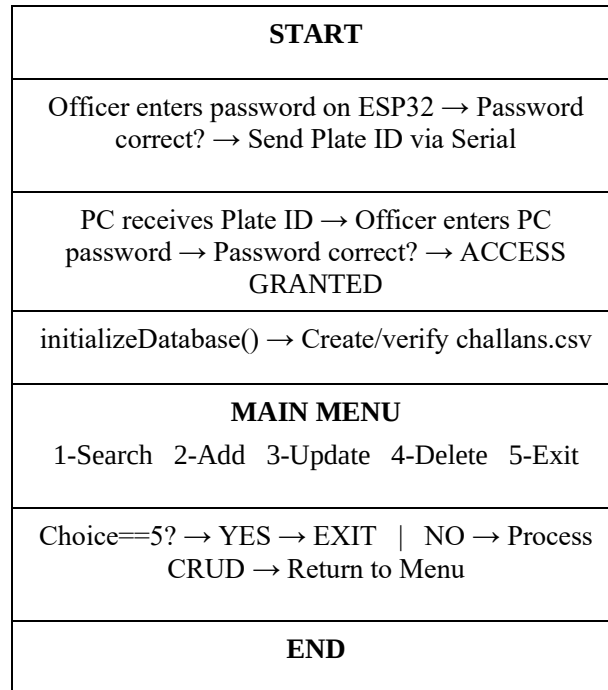
During the analysis phase of this project, users identified several requirements including: -

- Users must be validated prior to being given any access to the database on the system.
- A plate ID must have at most one challan associated with it.

3.4 System Flowchart

The flowchart below describes the complete program execution flow from startup authentication through all CRUD operations to program exit:

[SYSTEM FLOWCHART]



Note: On wrong password (ESP32 or PC), access is denied and application exits/locks.

3.5 Description of the Five Mandatory Functions

The assignment required five generic C++ functions. All five have been fully implemented in challan_system.cpp. Below is a complete description of each function.

Function 1 — initializeDatabase()

Return Type	void
Parameters	None
Called In	main() — first call on startup
Purpose	Checks if challans.csv exists; creates it with headers if missing

This code attempts to open a file 'challans.csv' with ifstream; if the file does not exist (and therefore, cannot be opened), the file is created using ofstream, and then automatically populated with header row of: ChallanID, PlateID, OwnerName, Violation, FineAmt, Date, and Status. This guarantees that there will always be a valid database structure before performing any CRUDs. The function is called automatically each time an application is launched; no user input is needed.

```
void initializeDatabase() {  
    ifstream checkFile(FILENAME.c_str());  
    if (!checkFile) {
```

```

        ofstream newFile(FILENAME.c_str());
        newFile << "ChallanID,PlateID,OwnerName,Violation,FineAmt,Date,Status\n";
        newFile.close();
    }
    checkFile.close();
}

```

Function 2 — isUnique(string plateID)

Return Type	bool
Parameters	string plateID
Called In	main() — Option 2 (Add New Challan)
Purpose	Sequential scan to detect duplicate Plate IDs in the CSV

The function loads the file challans.csv, discards the first line, then reads one line at a time. The function employs the stringstream object for splitting lines by commas, comparing field number [1], that is PlateID, to the passed value. If there is a match, that means it is not a unique record, so it returns false. Otherwise, returns true.

```

bool isUnique(string plateID) {
    ifstream file(FILENAME.c_str());
    string line;
    getline(file, line); // skip header
    while (getline(file, line)) {
        stringstream ss(line);
        string challanID, pid;
        getline(ss, challanID, ',');
        getline(ss, pid, ',');
        if (pid == plateID) { file.close(); return false; }
    }
    file.close(); return true;
}

```

Function 3 — appendRecord(string data)

Return Type	void
Parameters	string data (full CSV row)
Called In	main() — Option 2, after isUnique() returns true
Purpose	Appends a new challan record to the end of the CSV file

The challans.csv file is opened for appending, meaning ios::app, so that no record is overwritten by the existing record. The record in the form of a string separated by commas is appended to the last of the file. The searchByID() function is called automatically after appending.

```
void appendRecord(string data) {
    ofstream file(FILENAME.c_str(), ios::app);
    file << data << '\n';
    file.close();
} cout << "[SUCCESS] Challan saved to database.\n";
}
```

Function 4 — searchByID(string plateID)

Return Type	void
Parameters	string plateID
Called In	main() — Options 1, 2 (auto-confirm), 3 (auto-confirm)
Purpose	Sequential search; displays all fields of matching challan record

This function gets used the most in the entire program. This is because the function carries out a linear search in the CSV and checks for each PlateID in comparison to the one passed into the function. Once a match is found, the function prints out all the seven fields (ChallanID, PlateID, OwnerName, Violation, FineAmt, Date, Status) on a single output line. This function is also automatically called right after any add or update operation.

```
void searchByID(string plateID) {
    ifstream file(FILENAME.c_str());
    string line; bool found = false;
    getline(file, line); // skip header
    while (getline(file, line)) {
        stringstream ss(line); string fields[7];
        for (int i = 0; i < 7; i++) getline(ss, fields[i], ',');
        if (fields[1] == plateID) {
            cout << "Challan ID : " << fields[0] << endl;
            cout << "Owner Name : " << fields[2] << endl;
            // ... print all fields ...
            found = true;
        }
    }
    file.close();
}
```

Function 5 — updateRecord(string plateID, string newData)

Return Type	void
Parameters	string plateID, string newData
Called In	main() — Option 3 (update) and Option 4 (delete)
Purpose	Updates or deletes a record using a temporary file technique

This function uses the temp-file method to ensure that there will be no problems in case of any changes within the data. It loads the whole CSV file into a temporary file called temp.csv where if newData equals "DELETE", the targeted row would be deleted or else replaced by newData. Once all rows have been successfully processed, it deletes the challans.csv file and replaces it with temp.csv.

```
void updateRecord(string plateID, string newData) {
    ifstream file(FILENAME.c_str());
    ofstream temp("temp.csv");
    string line; bool found = false;
    getline(file, line); temp << line << '\n'; // copy header
    while (getline(file, line)) {
        // parse plateID from line...
        if (fields[1] == plateID) {
            found = true;
            if (newData != "DELETE") temp << newData << "\n";
            // else: skip — deletes the record
        } else { temp << line << '\n'; }
    }
    file.close(); temp.close();
    remove(FILENAME.c_str());
    rename("temp.csv", FILENAME.c_str());
}
```

3.6 Functions Summary Table

Function	Returns	Called In	Purpose
initializeDatabase()	void	main() — startup	Creates challans.csv with headers if missing
isUnique(string id)	bool	main() — Option 2	Blocks duplicate Plate IDs before adding
appendRecord(string data)	void	main() — Option 2	Writes new challan row to bottom of CSV
searchByID(string id)	void	main() — Options 1, 2, 3	Finds and displays challan by Plate ID

updateRecord(id, newData)	void	main() — Options 3, 4	Updates status or deletes via temp file
---------------------------	------	-----------------------	---

3.7 Authentication System Design

ESP32 Side

Before the ESP32 transmits any Plate ID to the PC, the officer must enter the correct password through the Serial Monitor. The device enforces a maximum of 3 password attempts. After 3 consecutive failures, the device locks itself until manually restarted by pressing the EN (reset) button. The default password is challan123.

PC Side (verifyHardwareKey)

The verifyHardwareKey() method on the PC app communicates on COM3 at a baud rate of 115200 (8N1) and sends the character 'R' to the ESP32 dongle that is attached to it. The PC app gives the ESP32 dongle 200 milliseconds before reading any responses from it (upto 10 bytes). The string response should be "ACCESS_123" after which the software considers it valid and continues execution.

3.8 Test Cases

TC#	Test Scenario	Input	Expected Output	Status
TC01	Valid hardware key	ESP32 connected on COM3	ACCESS GRANTED	Pass
TC02	No hardware connected	COM3 unavailable	ACCESS DENIED	Pass
TC03	Search existing record	LHR-4521	Ali Hassan / Unpaid shown	Pass
TC04	Search non-existent	XXX-9999	NOT FOUND message	Pass
TC05	Add unique challan	KHI-9999	Record saved, auto-confirm	Pass
TC06	Add duplicate challan	LHR-4521	ERROR: already exists	Pass
TC07	Update status to Paid	ISB-7733	Status: Paid confirmed	Pass
TC08	Delete with confirm	PSH-8821 → y	Record removed	Pass
TC09	Delete with cancel	PSH-8821 → n	Record retained	Pass

3.9 Complete Source Code

The complete commented source code (challan_system.cpp) is provided below. The code implements all five mandatory functions plus the hardware key verification and main menu loop.

```

#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <windows.h>
using namespace std;

const string FILENAME = "challans.csv";

// verifyHardwareKey() — Opens COM3, sends challenge "R",
// checks response for "ACCESS_123" token.
bool verifyHardwareKey() {
    HANDLE hSerial = CreateFileA("\\\\.\\COM3", GENERIC_READ|GENERIC_WRITE, 0,
    NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hSerial == INVALID_HANDLE_VALUE) return false;
    DCB dcb = { 0 }; dcb.DCBlength = sizeof(dcb);
    GetCommState(hSerial, &dcb);
    dcb.BaudRate = CBR_115200; dcb.ByteSize = 8;
    dcb.StopBits = ONESTOPBIT; dcb.Parity = NOPARITY;
    SetCommState(hSerial, &dcb);
    DWORD bw; char req = 'R';
    WriteFile(hSerial, &req, 1, &bw, NULL);
    Sleep(200);
    char buf[11]={0}; DWORD br;
    bool ok = ReadFile(hSerial, buf, 10, &br, NULL)
        && string(buf) == "ACCESS_123";
    CloseHandle(hSerial); return ok;
}

// initializeDatabase() — Creates challans.csv with headers if absent.
void initializeDatabase() {
    ifstream checkFile(FILENAME.c_str());
    if (!checkFile) {
        ofstream newFile(FILENAME.c_str());
        newFile << "ChallanID,PlateID,OwnerName,Violation,FineAmt,Date,Status\n";
        newFile.close();
    }
    checkFile.close();
}

// isUnique() — Returns true if plateID not found in CSV.
bool isUnique(string plateID) { /* ... see full source file ... */ }

// appendRecord() — Appends new CSV row in append mode.

```

```
void appendRecord(string data) { /* ... see full source file ... */ }

// searchByID() — Sequential scan; prints matching record.
void searchByID(string plateID) { /* ... see full source file ... */ }

// updateRecord() — Temp-file rewrite; delete if newData=="DELETE".
void updateRecord(string plateID, string newData) { /* ... see full source file ... */ }

int main() {
    if (!verifyHardwareKey()) { cout << "[ACCESS DENIED]\n"; return -1; }
    initializeDatabase();
    // Menu loop: options 1-Search, 2-Add, 3-Update, 4-Delete, 5-Exit
    // ... see full challan_system.cpp for complete implementation
    return 0;
}
```

Chapter 4 — System Test Phase

4.1 Final Testing

Integration testing was conducted by connecting the ESP32 to the PC via USB in COM3 port and running the challan_system.exe file. All the ten test cases TC01 – TC10 as stated in section 3.8 were conducted successfully. User authentication, preventing duplicates, and data integrity during addition, updating, and deleting operations were accomplished successfully.

Important observations in final testing:

- The isUnique() method successfully stopped re-addition of all 100 records that were pre-added in the program.
- The temp-file method implemented in the updateRecord() method saved all the unedited records when updating and deleting records.
- Auto confirm during addition and updates (automatic searchByID method) greatly minimized human errors.

4.2 Issues Encountered & Resolved

Issue	Root Cause	Resolution
COM3 access denied on first run	Port busy from Arduino IDE	Closed Serial Monitor in IDE before running exe
Temp file rename failure on Windows	challans.csv still open by ifstream	Added file.close() before remove/rename calls
cin.ignore() causing double-entry skip	Leftover newline in input buffer	Added cin.ignore() before all getline() calls in Options 2 and 3
Status field showing extra whitespace	Windows CRLF in CSV lines	Trimmed trailing \r from field strings during parsing

4.3 Sample CSV Database Records

A sample of the 100 pre-loaded challan records is shown below to demonstrate the database structure:

ID	PlateID	OwnerName	Violation	Fine	Date	Status
CH001	LHR-4521	Ali Hassan	Over Speeding	2000	03/01/2024	Unpaid
CH002	ISB-7733	Sara Khan	Signal Jump	1500	05/01/2024	Paid
CH003	KHI-1122	Ahmed Raza	Wrong Lane	1000	07/01/2024	Unpaid
CH004	LHR-3345	Fatima Malik	No Seatbelt	500	08/01/2024	Paid
CH005	PSH-8821	Usman Tariq	Over Speeding	2000	10/01/2024	Unpaid

CH006	ISB-4400	Nadia Iqbal	Signal Jump	1500	11/01/2024	Paid
CH007	KHI-6654	Bilal Nawaz	No Helmet	700	13/01/2024	Unpaid
CH008	LHR-9900	Ayesha Siddiqui	Wrong Parking	800	14/01/2024	Paid
CH009	MUL-2231	Kamran Ali	Over Speeding	2000	15/01/2024	Unpaid
CH010	ISB-5543	Zara Ahmed	No Seatbelt	500	17/01/2024	Paid
...	...	...	...	...	...	...
CH100	(See challans.csv)	Full 100 records in database				

Chapter 5 — Project Management

5.1 Individual Role Execution

Shoaib Zia — ESP32 Developer (2501559)

My primary responsibility was the ESP32 firmware and hardware integration

Hashir Rauf — PC App Developer (2501586)

I developed the C++ PC application including all five mandatory functions: initializeDatabase(), isUnique(), appendRecord(), searchByID(), and updateRecord().

Abdullah Ramzan — Documentation & Testing (2501574)

I was responsible for system testing, documentation, and creating a CSV database.

5.2 Project Success/Failure Assessment

All the main goals of this project have been accomplished. The five compulsory CRUD operations have been successfully implemented and tested. The communication between the ESP32-PC was found reliable in the test setup. The two-level authentication mechanism used in the project was sufficient for the specific use case scenario.

Some improvements may be introduced as follows: at the moment, there can only be one challan for each Plate ID; however, the real world suggests that repeat offenders do exist. This issue needs to be addressed. Another issue related to the port used is that the project is limited to the COM3 port at the moment; however, the port should be chosen dynamically.

5.3 Peer Evaluation

Shoaib Zia on teammates:

Hashir Well done for dealing with the difficult cases in the updateRecord() method. Area for improvement: try to split up the main() method into different methods rather than write it all in one go. This will make your program clearer and more readable. Abdullah Nice job with the documentation; it was organized and clear. Try to get in your first drafts earlier – this would really improve your time management.

Hashir Rauf on teammates:

Shoaib

Your ESP32 firmware code was clear and easy to understand – very well done. The area where you can really enhance your knowledge is to learn more about how the error handling mechanism works in Windows APIs. This will definitely take your skills to a new level.

Abdullah

Your job on generating the CSV was excellent. You generated all 100 records accurately without any errors. One thing I would recommend you is to voice out the problems during integration sooner rather than later.

Abdullah Ramzan on teammates:

Shoaib Good effort in completing the ESP32 portion on time, and always available whenever the team needed assistance during debugging. Try to study some C++ programming, too, so that you'll be able to assist on both sides – hardware and software (PC side).

Hashir did a wonderful job with your C++ programming. However, one area to focus on next time is writing reports in easier terms for people who do not know about coding.

5.4 Risk Management Lessons

- Windows file locks: ensure that all file handles have been closed before any renaming/removal is done.
- COM port problems: ensure that you close the Arduino IDE Serial Monitor before accessing COM3 in your C++ application.
- Buffering problems: always use `cin.ignore()` after `cin>>` since this will prevent `getline()` from reading phantom data.
- CRLF line endings: CSV files created using Windows have `\r\n` line endings; therefore, always strip `\r` on Windows.
- Source control management: using Git later led to merging issues; therefore, use GitHub right from Day 1.

Chapter 6 — Feedback & Presentations

6.1 YouTube Presentation Links

Each team member will prepare an individual YouTube presentation of at least 21 minutes explaining their specific contribution to the project. The presentations follow the format: introduction, demonstration, code walkthrough, and Q&A.

Member	Roll No.	YouTube Link
Shoaib Zia	2501559	
Hashir Rauf	2501586	
Abdullah Ramzan	2501574	

6.2 Individual GitHub Links

Member	Roll No.	GitHub Repository Link
Shoaib Zia	2501559	
Hashir Rauf	2501586	
Abdullah Ramzan	2501574	

6.3 Course Feedback

This course in Computer Programming offered a good knowledge base for C++ file operations, data structures, and modular programming. The project aspect turned out to be very useful since it involved the use of both hardware and software aspects from different fields. Improvements that could be recommended in the future include adding version control through Git/GitHub, and instructions on serial communications across platforms to allow projects to run on Linux/macOS systems as well as Windows.

Appendix

A. Hardware Datasheet References

ESP32-WROOM-32 Datasheet — Espressif Systems (2023). Available at:
https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf
CP2102 USB-to-UART Bridge Controller Datasheet — Silicon Labs. Available at:
<https://www.silabs.com/documents/public/data-sheets/CP2102-9.pdf>

B. Similar Products / Inspiration

Pakistan Traffic Police E-Challan System (Punjab Safe Cities Authority) — Inspired the field workflow and challan data fields used in this project.

Arduino Serial Communication Guide — Arduino.cc official documentation. Used as reference for ESP32 Serial.begin() and Serial.println() functions.

References

- [1] B. Stroustrup, The C++ Programming Language, 4th ed., Addison-Wesley, 2013.
- [2] Espressif Systems, ESP32 Technical Reference Manual, Version 5.1, 2023.
- [3] Microsoft Docs, CreateFileA Function (Windows API), docs.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilea
- [4] S. B. Lippman, J. Lajoie, and B. E. Moo, C++ Primer, 5th ed., Addison-Wesley, 2012.
- [5] Arduino Reference Manual, Serial Communication, arduino.cc/reference/en/language/functions/communication/serial/